



Formação Backend: Python & FastAPI — O Caminho Sannin de Orochimaru

Projeto Mushi Bingo

Aula Anterior

- Revisão de Banco de Dados (modelagem, normalização, SQL básico)
- Boas práticas de nomenclatura de banco de dados
- Apresentação do projeto: diagrama ER, regras de negócio e endpoints sugeridos
- Estrutura de pastas do projeto

Recapitulando: ORM, SQLAlchemy e Alembic

- **ORM:** manipular tabelas e registros como classes e objetos Python
- **SQLAlchemy:** o ORM que vamos utilizar no projeto
- **Alembic:** ferramenta de migrations, integrada ao SQLAlchemy

Hoje vamos sair da teoria e conectar o projeto a um banco de dados de verdade.

Configurando o SQLAlchemy

Criamos o `engine` (conexão com o banco) e a `Session` (unidade de trabalho para consultas e alterações).

```
# app/database.py

from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = "sqlite:///./database.db"

engine = create_engine(DATABASE_URL)
Session = sessionmaker(bind=engine)

Base = declarative_base()
```

Criando os Models: Autor e Estudio

```
# app/models/anime.py

from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base

class Autor(Base):
    __tablename__ = "autor"

    id: Mapped[int] = mapped_column(primary_key=True)
    nome: Mapped[str] = mapped_column(String(100), unique=True)

    animes: Mapped[list["Anime"]] = relationship(back_populates="autor")

class Estudio(Base):
    __tablename__ = "estudio"

    id: Mapped[int] = mapped_column(primary_key=True)
    nome: Mapped[str] = mapped_column(String(100), unique=True)

    animes: Mapped[list["Anime"]] = relationship(back_populates="estudio")
```

Criando o Model: Anime

```
# app/models/anime.py (continuação)

from datetime import datetime

from sqlalchemy import ForeignKey, String

class Anime(Base):
    __tablename__ = "anime"

    id: Mapped[int] = mapped_column(primary_key=True)
    nome: Mapped[str] = mapped_column(String(150))
    episodios: Mapped[int]
    nota: Mapped[float | None]
    status: Mapped[str]
    capa: Mapped[str | None]
    criado_em: Mapped[datetime] = mapped_column(default=datetime.now)
    atualizado_em: Mapped[datetime] = mapped_column(
        default=datetime.now, onupdate=datetime.now
    )

    autor_id: Mapped[int | None] = mapped_column(ForeignKey("autor.id"))
    estúdio_id: Mapped[int | None] = mapped_column(ForeignKey("estudio.id"))

    autor: Mapped[Autor | None] = relationship(back_populates="animes")
    estúdio: Mapped[Estudio | None] = relationship(back_populates="animes")
```

Alembic: Configuração Inicial

```
uv add alembic  
  
uv run alembic init migrations
```

Em `migrations/env.py`, aponte o `target_metadata` para os models do projeto:

```
from app.database import Base  
from app.models.anime import Autor, Estudio, Anime # noqa  
  
target_metadata = Base.metadata
```

Dica: Ajuste também `sqlalchemy.url` em `alembic.ini` para

```
sqlite:///./database.db
```

Alembic: Gerando e Aplicando Migrations

```
# Gera o script de migration comparando os models com o banco  
uv run alembic revision --autogenerate -m "cria tabelas autor, estudio e anime"
```

```
# Aplica a migration, criando as tabelas de fato  
uv run alembic upgrade head
```

Dica: Sempre revise o arquivo gerado antes de aplicar a migration.

Organizando rotas com APIRouter

Até agora todos os endpoints viviam em `main.py`. O `APIRouter` permite dividir as rotas por recurso.

```
# app/routers/animés.py

from fastapi import APIRouter

router = APIRouter(prefix="/animés", tags=["animés"])
```

```
# app/main.py

from fastapi import FastAPI
from app.routers import animés

app = FastAPI()

app.include_router(animés.router)
```

Persistindo os dados no banco

O `database = []` sai de cena — agora os endpoints usam a `Session` do SQLAlchemy.

```
# app/routers/animes.py

from http import HTTPStatus

from app.database import Session
from app.models.anime import Anime
from app.schema.anime import AnimeSchema, AnimePublic

@router.post("/", status_code=HTTPStatus.CREATED, response_model=AnimePublic)
def create_anime(anime: AnimeSchema):
    with Session() as session:
        db_anime = Anime(**anime.model_dump())
        session.add(db_anime)
        session.commit()
        session.refresh(db_anime)
```

Persistindo os dados no banco (GET)

```
# app/routers/animés.py

from app.schema.anime import AnimeList

@router.get("/", response_model=AnimeList)
def read_animés():
    with Session() as session:
        animés = session.query(Anime).all()

        return {"animés": animés}
```



Atividade

- Siga o mesmo padrão de `/animés` para implementar `/autores` e `/estúdios`
- Teste os endpoints pelo Swagger (`/docs`) e confira se os dados estão sendo salvos no `database.db`
- Fique livre para criar filtros ou endpoints extras que fizerem sentido para o seu projeto

Próximos tópicos

- Injeção de Dependência (Depends)
- RedirectResponse
- Middleware
- Tratamento de exceções mais robusto

Referencias

- [SQLAlchemy — ORM Quick Start](#)
- [SQLAlchemy — Declarative Mapping](#)
- [Alembic](#)
- [FastAPI — Bigger Applications \(APIRouter\)](#)
- [FastAPI do Zero](#)



Até a próxima!